

React JS

Introdução a frameworks e bibliotecas

Material

MOBILE_003



Frameworks X Bibliotecas

Diferença entre frameworks e bibliotecas





Agenda

- Diferença entre frameworks e bibliotecas
- Benefícios de usar frameworks e bibliotecas no desenvolvimento do lado do cliente
- Visão geral dos frameworks e bibliotecas populares
- Foco na introdução ao React JS
- Os principais conceitos do React JS



Biblioteca

- Uma biblioteca é um conjunto de funções e utilitários pré-escritos que ajudam os desenvolvedores a realizar tarefas específicas de forma mais eficiente.
- As bibliotecas são flexíveis e permitem que os desenvolvedores escolham quais recursos desejam utilizar.



Framework

- Um framework, por outro lado, é uma estrutura abrangente que define uma arquitetura e um conjunto de regras para desenvolver aplicativos.
- Os frameworks têm uma abordagem mais prescritiva, fornecendo uma estrutura completa para o desenvolvimento, com diretrizes sobre como estruturar e organizar o código.

Benefícios

Benefícios ao usar
frameworks e
bibliotecas no
desenvolvimento do
lado do cliente



Benefícios

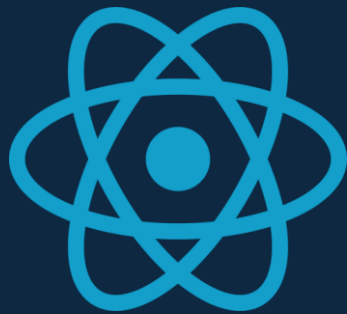
- **Reutilização de código:** frameworks e bibliotecas fornecem componentes ou módulos reutilizáveis que podem ser utilizados em diferentes partes do aplicativo, economizando tempo e esforço no desenvolvimento.
- **Produtividade aprimorada:** essas ferramentas simplificam tarefas comuns, como manipulação do DOM, gerenciamento de estado e manipulação de eventos, permitindo que os desenvolvedores se concentrem na lógica de negócios em vez de lidar com detalhes de implementação.
- **Manutenção facilitada:** frameworks e bibliotecas geralmente seguem padrões de projeto e boas práticas, o que torna o código mais organizado e de fácil manutenção, mesmo em projetos complexos.
- **Comunidade ativa:** muitas frameworks e bibliotecas possuem uma comunidade ativa de desenvolvedores que contribuem com recursos, documentação, tutoriais e suporte, tornando mais fácil aprender e resolver problemas.

Visão geral dos frameworks e bibliotecas populares

- Existem várias opções populares de frameworks e bibliotecas para o desenvolvimento do lado do cliente. Alguns exemplos são:
 - **React JS:** uma biblioteca JavaScript de código aberto mantida pelo Facebook, focada na criação de interfaces de usuário interativas e reativas.
 - **Angular:** um framework TypeScript mantido pelo Google, que oferece recursos abrangentes para desenvolvimento de aplicativos web escaláveis e complexos.
 - **Vue.js:** uma biblioteca JavaScript progressiva e fácil de aprender, que permite criar interfaces de usuário dinâmicas e reativas.

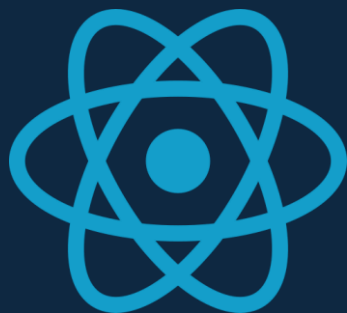
Outros

- Além dos vistos, temos o **Svelte**, **Next.js** e **Remix** que são frameworks modernos que oferecem diferentes abordagens para o desenvolvimento web com React.
 - **Svelte** foca em desempenho e simplicidade,
 - **Next.js** em recursos para SEO e desenvolvimento otimizado, enquanto o
 - **Remix** se destaca pela sua aderência aos web standards e experiência de usuário aprimorada.



React JS

- É uma biblioteca **JavaScript** popular e amplamente adotada para a criação de interfaces de usuário reativas e eficientes.
- Com base em componentes reutilizáveis, o **React** permite criar aplicativos web interativos, de fácil manutenção e de alto desempenho.



React Native

- É um framework de código aberto desenvolvido pela Meta que permite criar aplicativos móveis nativos (para Android e iOS) utilizando JavaScript e os mesmos princípios de design do React.
- Em vez de renderizar visualizações na web, o React Native utiliza componentes nativos da plataforma, permitindo que os aplicativos tenham uma experiência de usuário fluida e um desempenho próximo ao de um aplicativo desenvolvido nativamente com Swift, Objective-C, Java ou Kotlin, mas com a vantagem de manter uma única base de código (codebase) para ambas as plataformas.

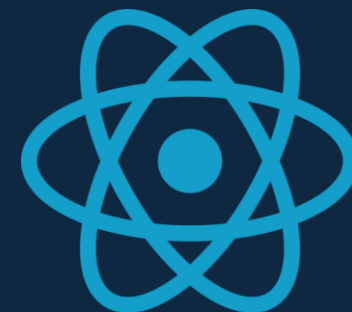


Vite

- O **Vite** é atualmente a ferramenta de construção (build tool) preferida da comunidade para projetos React e React Native, substituindo ferramentas legadas como o **Create React App** por ser extremamente rápido e moderno.
- Usaremos em nossos exemplo React.

Os principais conceitos do React JS

Material
MOBILE_003



Conceitos

- **Componentes:** unidades independentes de código que encapsulam a lógica e a renderização de partes específicas da interface do usuário.
- **JSX:** uma extensão de sintaxe que permite escrever HTML no JavaScript, facilitando a definição da estrutura da interface do usuário.
- **Virtual DOM:** uma representação virtual eficiente do DOM real, que o React usa para otimizar as atualizações da interface do usuário e melhorar o desempenho.
- **Hooks:** como useState e useEffect
- **Gerenciadores de estado:** Para aplicações complexas, apresentar brevemente gerenciadores de estado como Redux e Zustand.

Nota

- Ao usar o React JS, você pode criar componentes reutilizáveis e compor uma interface de usuário complexa a partir deles.
- A biblioteca gerencia eficientemente as atualizações do DOM, minimizando as operações de manipulação direta, resultando em um desempenho aprimorado.



Laboratório

- Desenvolvimento de uma aplicação SPA usando o React JS

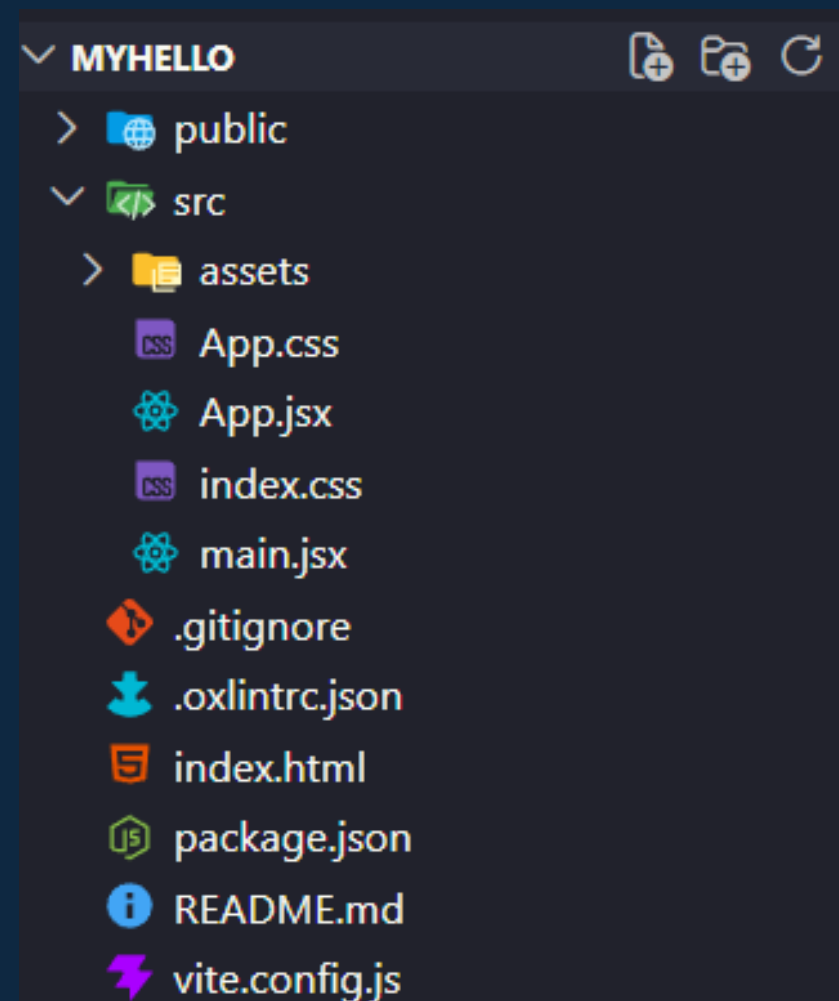


Passo 1 – Configuração do ambiente

1. Certifique-se de ter o Node.js instalado em seu computador.
2. Abra o terminal ou prompt de comando e navegue até o diretório onde deseja criar o projeto.
3. Execute o comando `npm create vite@latest myhello -- --template react` para criar um novo projeto React chamado "myhello". Aguarde até que a criação do projeto seja concluída.
4. O processo anterior, de criação do projeto, pode levar alguns instantes enquanto é montado todo o boilerplate mínimo que uma aplicação React para a web precisa ter.

Passo 2 – Estrutura do projeto React

1. Abra o diretório do projeto criado no passo anterior.
[**cd myhello**]
2. Explore a estrutura do projeto, incluindo os diretórios "src" e "public" e os arquivos "main.jsx" e "App.jsx".
3. Abra no VS Code [**code .**]
4. Execute o comando **npm install**.



Estrutura

- Dentro do projeto você irá encontrar três pastas, cada qual terá uma série de arquivos que é utilizado de alguma forma pela aplicação React:
 - **node_modules** - Pasta responsável por guardar todas as dependências do projeto. Essa pasta é criada pelo NPM ou pelo Yarn no momento em que uma biblioteca, como o próprio React, é adicionada ao projeto.
 - **public** - Pasta pública da aplicação web. Nela ficam guardados os arquivos iniciais da aplicação, como o index.html que é interpretado pelo navegador, o ícone do site e um arquivo JSON com os parâmetros de configuração do site.
 - **src** - Código fonte da aplicação: Aqui ficarão todos os arquivos JavaScript que irão compor a nossa aplicação.

Passo 3 – Verifique o arquivo main.jsx

```
● ● ●  
  
import { StrictMode } from 'react'  
import { createRoot } from 'react-dom/client'  
import './index.css'  
import App from './App.jsx'  
  
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <App />  
  </StrictMode>,  
)
```

Passo 4 – Altere o arquivo index.css

```
● ● ●  
  
.App {  
  text-align: center;  
  padding: 20px;  
}  
h1 {  
  font-size: 3rem;  
  background-color: springgreen;  
  border-radius: 10px;  
  padding: 15px;  
}
```

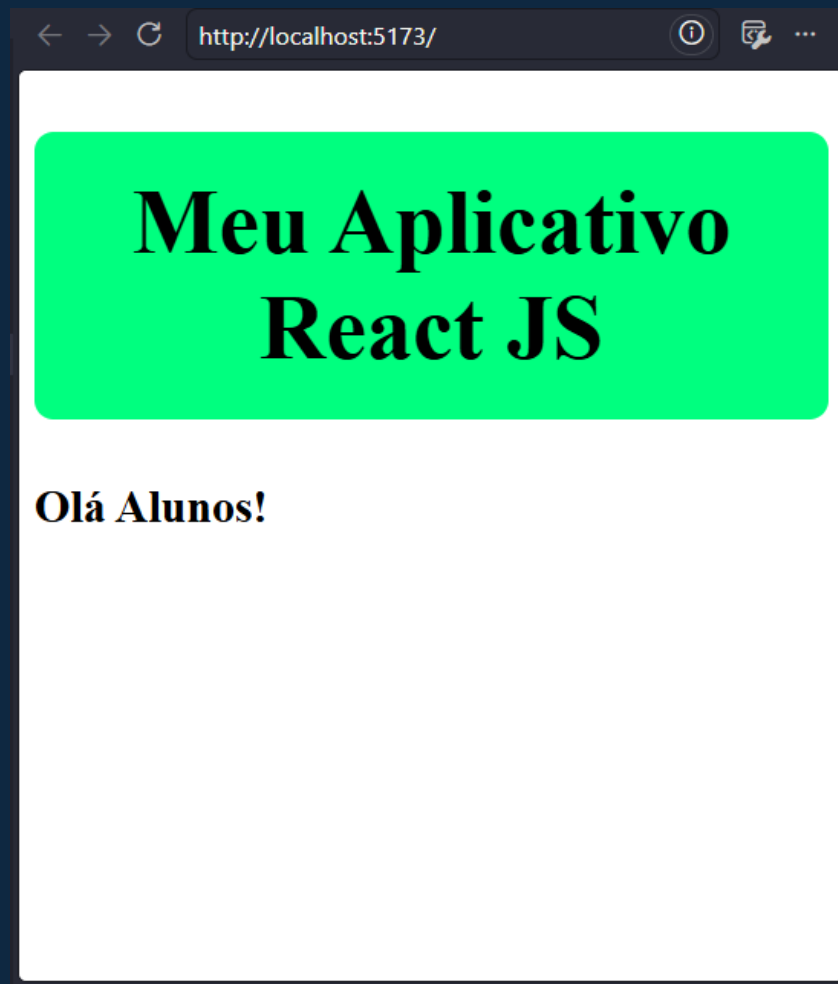
Passo 5 – no diretório src

Arquivo App.jsx

```
import './index.css'

export default function App() {
  return (
    <div>
      <h1 className="App">Meu Aplicativo React JS</h1>
      <h2>Olá Alunos!</h2>
    </div>
  )
}
```

Resultado da Execução



React JS

Criando Componentes

Material

MOBILE_003





Agenda

1. Criar Componentes
2. Exportar Componentes
3. Importar Componentes
4. Estilizar e Organizar Componentes

Componentes

- Componentes representam fundamentalmente elementos visuais que são apresentados ao usuário na interface da tela.
- Considerando, por exemplo, uma página Web, cada elemento de interação, seja um botão, um item do menu ou até mesmo um campo para edição de dados, são definidos como componentes.
- Além disso, é importante salientar que a página Web como um todo, com todas as suas partes interconectadas, também é caracterizada como um componente - este que abriga seus próprios componentes filhos.

Componentes no React JS

- No universo do React, cada elemento visual é considerado um componente.
- A adoção de componentes se mostra extremamente útil para a manutenção da aplicação, visto que permite a divisão de um código complexo em partes mais digeríveis e menores, facilitando a leitura e a gestão.
- Outro benefício notável é a possibilidade de reutilizar nosso código, maximizando a eficiência do desenvolvimento.

Pais e Filhos

- A tarefa de desenvolver uma aplicação em React implica essencialmente na concepção dos primeiros componentes 'filhos'.
- Posteriormente, esses componentes são 'importados' e integrados ao componente 'pai' principal - o componente fundamental que representa a totalidade da página. Vale salientar que, neste exemplo, estamos tratando de uma aplicação que consiste em uma única página.

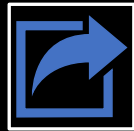
Mecânica de uma Aplicação React

1. Primeiro você vai codificar o seu componente (por exemplo, um botão na tela);
2. Com o componente codificado, o segundo passo é exportar esse componente - o que significa deixá-lo exposto para que possa ser usado na sua página;
3. Por último você vai 'importar' o componente previamente exposto e dessa forma apresentá-lo na tela.

Em termos mais técnicos



Criar o componente filho;



Exportar o componente filho;



Importar o componente filho dentro do componente pai (neste exemplo, na própria página).



1

Criar Componente Filho



Componentes

- Componente Header
- Componente Card
- Componente Footer



2

Exportar o Componente Filho



Componente

Header

export





3

Importar o componente filho dentro do pai

Componente
Header

The diagram illustrates the export of a component header. On the left, a dark blue rounded rectangle contains the text 'Componente' in bold and 'Header' below it. An arrow labeled 'export' points from this rectangle to a light blue rounded rectangle on the right. The light blue rectangle contains the text 'Componente pai' at the top, 'App' in the middle, and 'import Header' at the bottom. A decorative blue arrow points from the left edge towards the 'Componente Header' box.

export

Componente pai

App

import
Header

Arquitetura de Exemplo

- É um exemplo simples, contendo apenas uma página Web com alguns componentes estáticos.
- Não há entrada de dados do usuário, apenas a exibição de componentes visuais, tais como topo, rodapé, card.
- A arquitetura deste exemplo consiste na mecânica de importação de componentes filho dentro de um componente pai principal, chamado App, como mostra a Figura.

Componentes
filhos

topo

rodapé

corpo

componentes
filho são
importados
dentro do
componente App,
que representa
a página



Laboratório

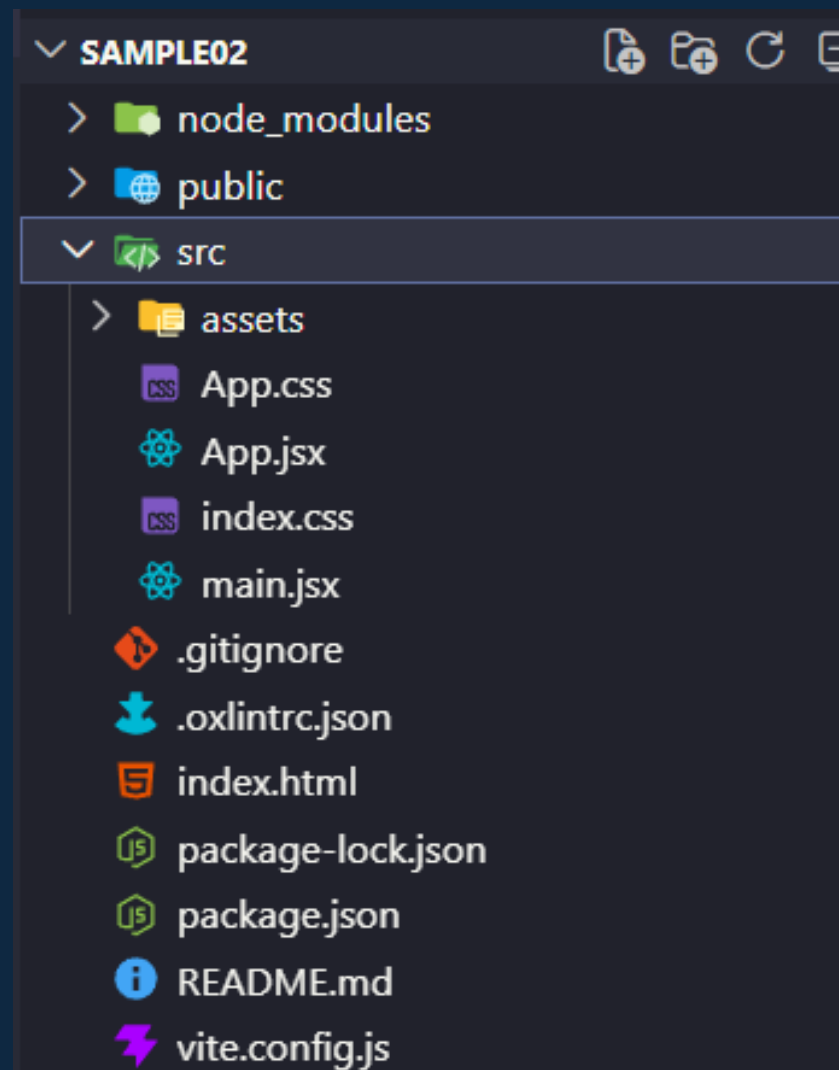
- Desenvolvimento de uma aplicação SPA usando o React JS. Nome da aplicação **sample02**.



Passo 1

- Execute o comando `npm create vite@latest sample02 -- --template react` para criar um novo projeto React chamado "sample02". Aguarde até que a criação do projeto seja concluída.
- O processo anterior, de criação do projeto, pode levar alguns instantes enquanto vite monta todo o boilerplate mínimo que uma aplicação React para a web precisa ter.
- Entre no diretório [`cd sample02`] e digite [`code .`] para abrir o VS Code.

Passo 2: Estrutura do projeto React



Antes de começarmos a criar nossos componentes filho como o Header, o Footer, o Card e até mesmo a página HomePage, precisamos lembrar que eles farão parte de um componente maior, como vemos na Figura.



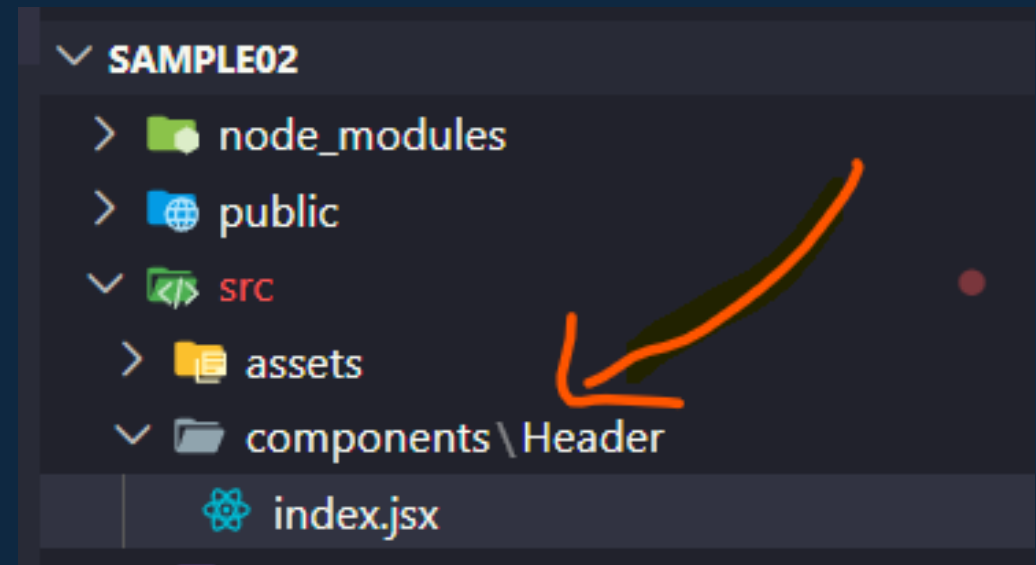
Passo 3: Preparando o componente pai

- Esse componente é chamado **App** e já foi criado junto com o projeto na aula anterior.
- O que precisamos fazer é remover o conteúdo que foi automaticamente criado dentro dele e inserir o nosso Código:
 - É o código estrutural do componente **App**. Por padrão esse é o primeiro componente a ser exibido na aplicação e será nosso componente pai.
 - Agora, com nosso componente pai pronto, podemos prosseguir e criar o primeiro componente filho, **Header**.

```
export default function App () {  
  return(  
    <div>  
      <h1 className="App">  
  
      </h1>  
    </div>  
  )  
}
```

Passo 4: Preparando os filhos Componente Header

- Com o nosso componente App criado, agora podemos começar a construir os componentes filhos que vão compor nossa página.
- Agora vamos criar o componente Header.
- Para organizar melhor os arquivos do nosso código vamos criar uma pasta chamada **'components'** dentro do diretório **src**.
- Dentro do diretório componentes vamos criar uma pasta chamada **Header** e nela um arquivo chamado **index.jsx**. Esse arquivo terá o código do nosso componente Header, como mostra a Figura.



Passo 4: Preparando os filhos

Componente Header (Altere o index.jsx)



```
export default function Header(){  
  return (  
    <header>  
      <h1>Trabalhando com os componentes</h1>  
    </header>  
  )  
}
```

Nota

- Ao darmos vida a um novo componente - ou seja, um elemento que será projetado na tela - é fundamental importar o componente React da dependência 'react'. Contudo, note que essa operação não é mais necessária, pois o processo se tornou automático.
- Adotar essa prática permite ao React interpretar corretamente o código, reconhecendo-o como um componente destinado a ser renderizado na tela. Isso faz parte da mecânica padrão no processo de construção de componentes utilizando React.

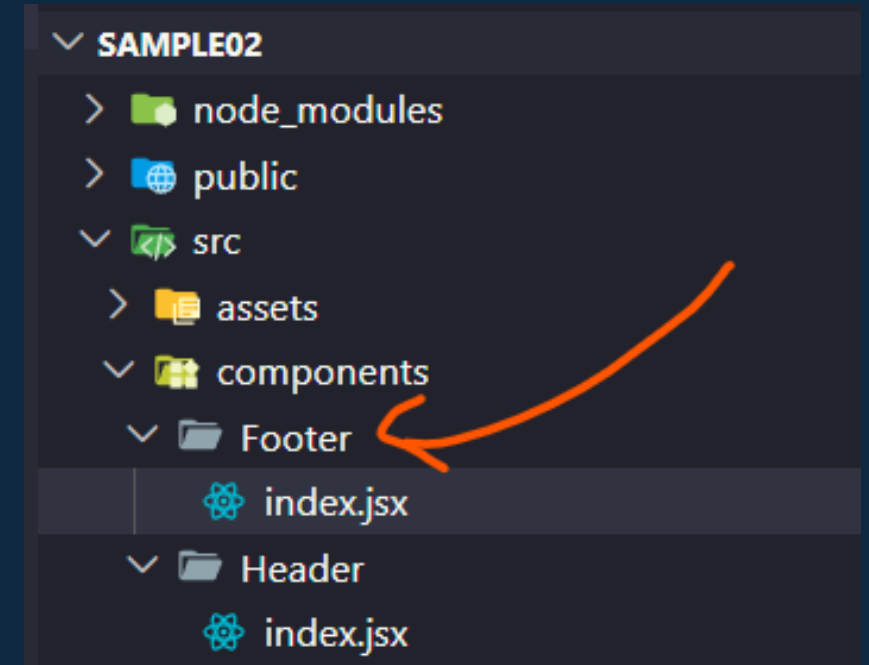
Nota 2

- Em seguida, observamos uma função JavaScript que **porta o mesmo nome do componente em questão** - este é um comportamento padrão na codificação de componentes em React.
- Todo componente é obrigado a retornar algo que será renderizado na tela. Este retorno é executado por meio da palavra-chave **'return'**. Dentro deste 'return', encontramos uma tag JSX <header> (veja a nota) com o texto "Trabalhando com componentes".
- Com o componente finalmente criado, é necessário exportá-lo para permitir que o componente App possa utilizá-lo. Esta exportação é realizada através do uso da palavra-chave **'export'**.

Passo 5: Preparando os filhos

Componente Footer

- Dentro do diretório **components** vamos criar uma pasta chamada **Footer** e nela um arquivo chamado **index.jsx**. Esse arquivo terá o código do nosso componente Footer, como mostra a Figura.



Passo 5: Preparando os filhos

Componente Footer

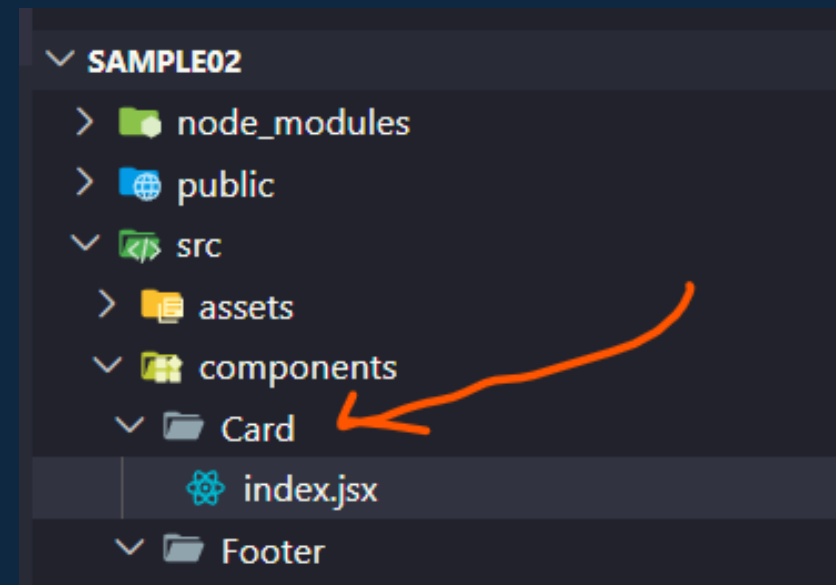
Pasta Footer, arquivo: index.jsx



```
export default function Footer () {  
  return(  
    <footer>  
      <p>Desenvolvido por Maromo</p>  
    </footer>  
  )  
}
```

Passo 6: Preparando os filhos Componente Card

- Dentro do diretório **components** vamos criar uma pasta chamada **Card** e nela um arquivo chamado **index.jsx**. Esse arquivo terá o código do nosso componente Card, como mostra a Figura.



Passo 6: Preparando os filhos

Componente Card

Pasta Card, arquivo: index.jsx



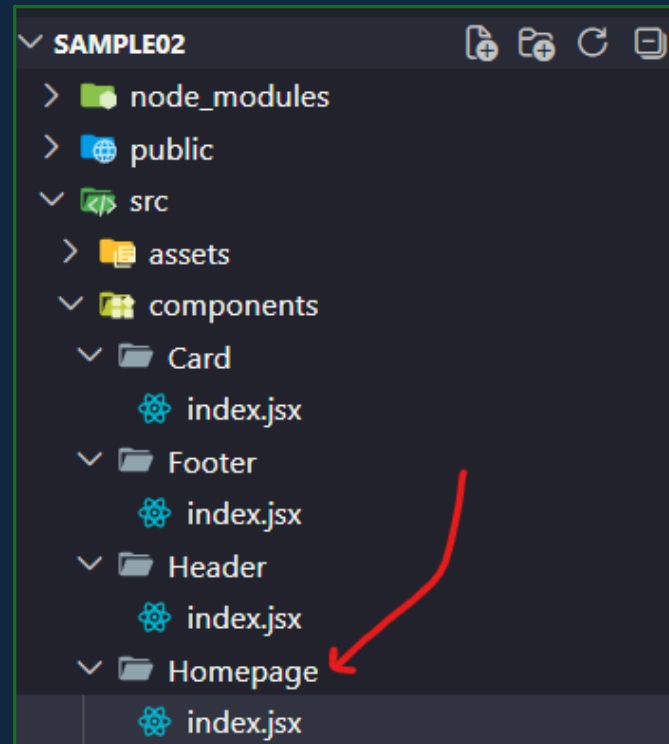
```
export default function Card(){
  return (
    <div className="card">
      <p>Componentes:</p>
      <p>Para facilitar partes menores</p>
      <p>Foco no reuso</p>
    </div>
  )
}
```

Passo 7: Importando componentes criados

- Com nossos dois componentes criados (**Header, Card e Footer**) chegou a hora de exibí-los no navegador.
- A princípio não vamos nos preocupar com a estilização e sim em aprender a parte lógica. No final deste curso vamos estilizar toda nossa aplicação.
- Agora vamos importar os componentes Header, Card e Footer para nosso componente principal App.

Passo 8: Criando noss HomePage Componente: HomePage - index.jsx

- Crie uma pasta com o nome **HomePage** e dentro um arquivo chamado **index.jsx** e insira o seguinte código no arquivo que acabamos de criar, como vemos no Código.



```
import Card from "../Card";

export default function Homepage(){
  return(
    <div>
      <h1>Homepage</h1>
      <Card />
      <Card />
      <Card />
    </div>
  )
}
```

Passo 9: Importando componente filho HomePage no arquivo principal App.jsx:

- Com nosso último componente criado só precisamos importá-lo no componente principal e dessa forma teremos o seguinte resultado apresentado na Figura.

```
import Homepage from "../components/Homepage"
import Header from "../components/Header"
import Footer from "../components/Footer"

export default function App () {
  return (
    <div>
      <h1 className="App">
        <Header />
        <Homepage />
        <Footer />
      </h1>
    </div>
  )
}
```

A man with dark, slightly messy hair is shown in profile, looking back over his right shoulder. He is wearing a black, heavy-duty jacket with a large, dark grey or olive green panel on the back, secured with several black straps and buckles. The jacket has a high collar and a visible zipper. The background is a plain, light-colored wall.

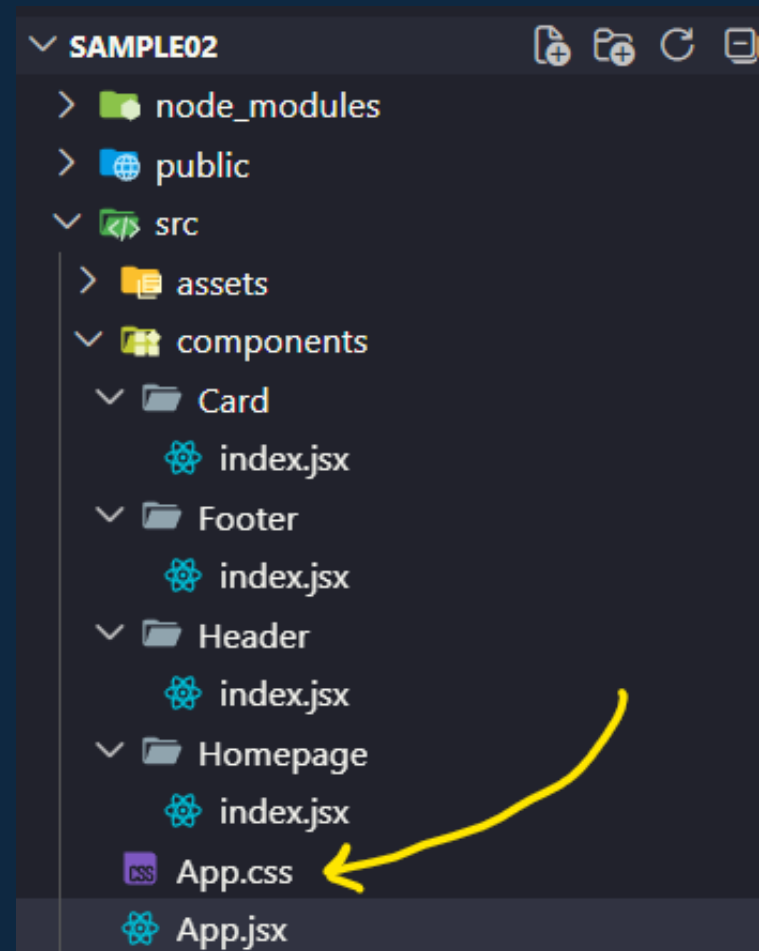
A circular graphic with the word "STYLE." repeated four times around the perimeter, set against a background of a window with a grid pattern.

Passo 10: ESTILIZANDO CSS

- Agora que temos a estrutura da nossa aplicação pronta, vamos melhorar a aparência dela.
- O primeiro passo é estilizar o componente pai **App**. Para isso vamos editar o arquivo **App.css**, que encontramos na estrutura apresentada na Figura.

```
import Homepage from "../components/Homepage"
import Header from "../components/Header"
import Footer from "../components/Footer"

import './App.css'
export default function App () {
  return (
    <div>
      <h1 className="App">
        <Header />
        <Homepage />
        <Footer />
      </h1>
    </div>
  )
}
```



Passo 10: ESTILIZANDO CSS

Arquivo App.css



```
.App {  
  display: flex;  
  flex-direction: column;  
  justify-content: space-between;  
  height: 100vh;  
}
```

Passo 10: ESTILIZANDO CSS

App.css

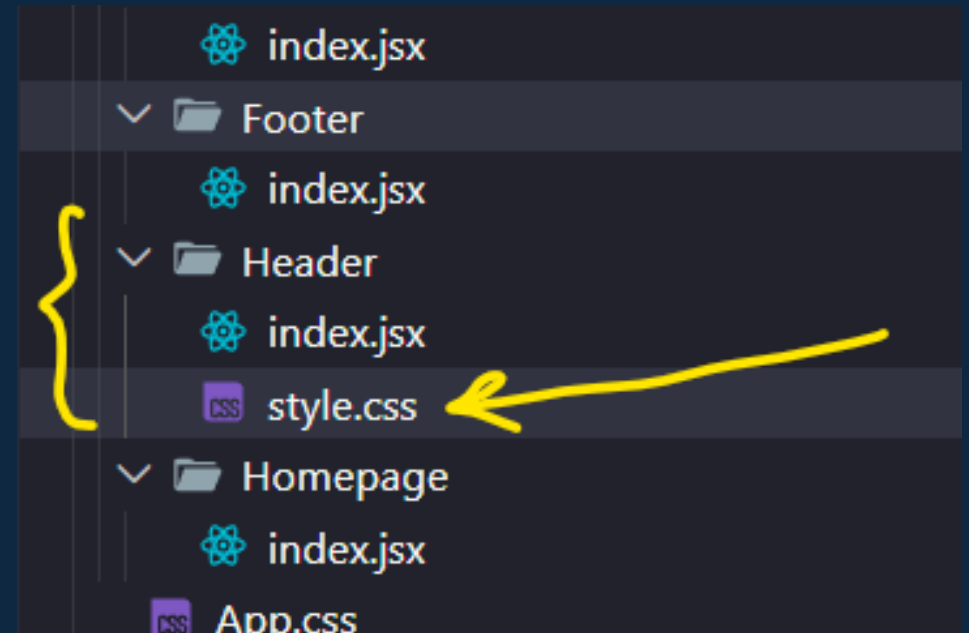
- O que temos aqui é o uso de display **flex** para organizar os elementos da página.
- Dizemos que **flex-direction** é column, pois queremos um elemento abaixo do outro (**Header** acima de **HomePage** e **HomePage** acima de **Footer**).
- O mais importante, para o entendimento do nosso código React é o seguinte: na primeira linha estamos identificando para o CSS que todo elemento que tiver a classe App será estilizado com as propriedades definidas dentro das chaves:

```
.App{
```

```
}
```

Passo 11: ESTILIZANDO CSS Header

- Para estilizar o componente **Header** vamos precisar criar um novo arquivo **.css**, pois ao contrário de App, este componente não possui um arquivo CSS previamente associado.
- Crie um arquivo chamado **style.css** na mesma pasta do componente Header, como vemos na Figura.



Passo 11: ESTILIZANDO CSS style.css (Header)

```
header {  
  margin: 5px;  
  background-color: #296067;  
  border-radius: 5px;  
  font-weight: 500;  
  font-size: 1.4rem;  
  color: #fff;  
  height: 60px;  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

```
import './style.css'  
  
export default function Header(){  
  return (  
    <header>  
      <h1>Trabalhando com os componentes</h1>  
    </header>  
  )  
}
```

altere o Header

Passo 12: ESTILIZANDO CSS

Card - alterando o index.jsx

- Para estilizar o componente Card vamos precisar criar um novo arquivo .css, pois ao contrário de App, este componente não possui um arquivo CSS previamente associado.
- Crie um arquivo chamado **style.css** na mesma pasta do componente **Card**, como vemos na Figura.

```
import './style.css'

export default function Card(){
  return (
    <div className="card-container">
      <p>Componentes:</p>
      <p>Para facilitar partes menores</p>
      <p>Foco no reuso</p>
    </div>
  )
}
```

Passo 12: ESTILIZANDO CSS style.css (Card)



```
.card-container {  
  border: 1px solid #296027;  
  border-radius: 7px;  
  width: 200px;  
  height: 200px;  
  background-color: #f1f1f1;  
  margin: 5px;  
  padding: 15px;  
}
```

Passo 13: ESTILIZANDO CSS Footer

- Para estilizar o componente **Footer** vamos precisar criar um novo arquivo .css, pois ao contrário de App, este componente não possui um arquivo CSS previamente associado.
- Crie um arquivo chamado **style.css** na mesma pasta do componente Header, como vemos na Figura.

```
import './style.css'

export default function Footer () {
  return(
    <footer>
      <p>Desenvolvido por Maromo</p>
    </footer>
  )
}
```

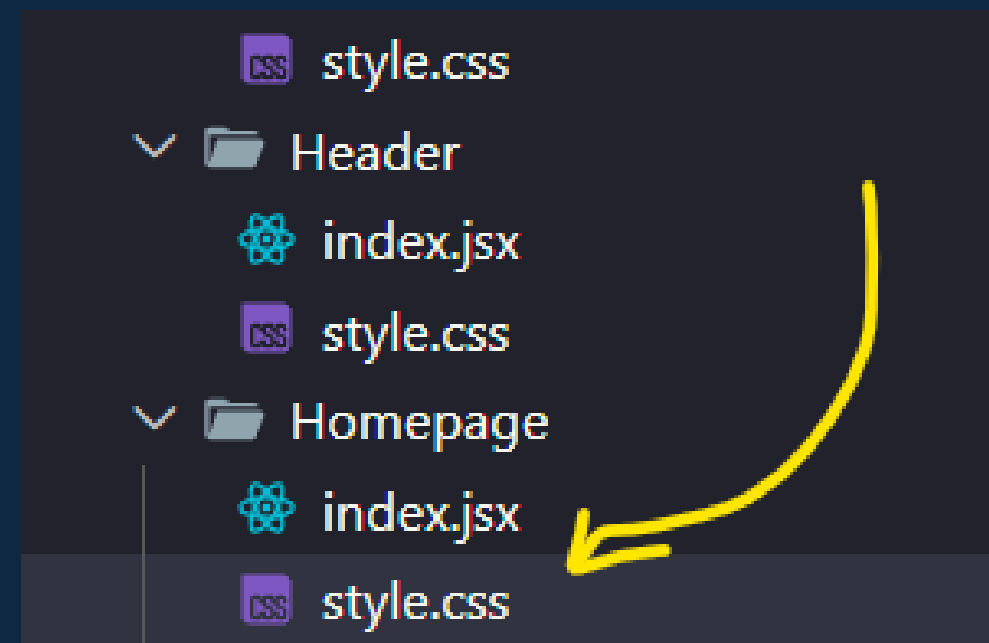
Passo 13: ESTILIZANDO CSS style.css (Footer)

```
● ● ●  
footer {  
  margin: 5px;  
  background-color: #296067;  
  font-weight: 500;  
  font-size: 1.4rem;  
  border-radius: 10px;  
  color: #fff;  
  height: 60px;  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

Passo 14: ESTILIZANDO CSS

Finalmente a Homepage

- Para estilizar o componente HomePage vamos precisar criar um novo arquivo .css, pois ao contrário de App, este componente não possui um arquivo CSS previamente associado.
- Crie um arquivo chamado **style.css** na mesma pasta do componente HomePage, como vemos na Figura



```
import Card from "../Card";

import "../style.css"
export default function Homepage(){
  return(
    <div className="homepage">
      <Card />
      <Card />
      <Card />
    </div>
  )
}
```

```
.homepage {
  font-size: 1rem;
  display: flex;
  justify-content: center;
  align-items: center;
  flex-wrap: wrap;
  width: 100vh;
  margin: 0 auto;
}
```

Passo 14: ESTILIZANDO CSS
style.css (HomePage)

Resultado Final

Trabalhando com os componentes

Componentes:

Para facilitar partes menores

Foco no reuso

Componentes:

Para facilitar partes menores

Foco no reuso

Componentes:

Para facilitar partes menores

Foco no reuso

Desenvolvido por Maromo

Conclusão

- No React, criamos componentes modulares e reutilizáveis para melhor manutenção do código.
- Exportamos e importamos esses componentes para utilização em diferentes partes do sistema, otimizando a eficiência do código.
- A estilização e organização apropriada dos componentes asseguram uma experiência de usuário eficaz e um código limpo.





Agora
você será
capaz de:

- **Compreender a Arquitetura de Software:** Diferenciar com clareza o funcionamento de uma biblioteca (focada em utilitários e flexibilidade, como o React JS) de um framework (estrutura opinativa e prescritiva, como o Angular).
- **Dominar a Componentização Modular:** Criar componentes funcionais e reutilizáveis utilizando JSX, aplicando o conceito de divisão de interfaces complexas em partes menores e de fácil manutenção.
- **Estruturar Aplicações React (Parent-Child):** Dominar o fluxo de importação e exportação de dados e elementos visuais, sabendo como aninhar componentes filhos dentro de um componente pai (App).
- **Organizar e Estilizar Projetos Modernos:** Trabalhar com a estrutura do Vite para web, criando folhas de estilo CSS dedicadas e isoladas para cada componente, garantindo um código limpo e escalável.

Fim



Prof. Maromo